Design Models: CDFG, FSM and Task Schedule

Gesture-Controlled Smart Glove for Human–Computer Interaction

BACSE324 · Embedded Systems Architecture · Fall 2026 · Review 2

Field	Value
Project title	Gesture-Controlled Smart Glove for Human–Computer Interaction
Scope tier	Tier 2 — full embedded system
Submission date	September 2026
Repository	`github.com/dayaj1222/data-glove`

Member	Reg. no.	Module owned / sections
Dayakrishn Joshi	25BCE2951	ESP-32 firmware, MPU-6500 I²C + flex ADC acquisition — §1, §2, §4
Syed Mohammad Ayaan	25BCE2311	ESP-32 wireless link, PC recognition pipeline — §3, §5, §6, §7

Diagrams are our own (Excalidraw and TikZ). Every timing figure is labelled measured, datasheet, or estimated.

1 System recap

The Smart Glove is a wearable input device that converts hand pose (two flex sensors) and hand motion (an MPU-6500 6-axis IMU) into standard keyboard and mouse events on a PC, for a single operator who wants hands-free control. The ESP-32 firmware samples both flex ADC channels and the IMU over I^2C at a fixed rate and frames them into a fixed-width binary record: five 16-bit flex counts followed by nine 32-bit floats (accelerometer x, y, z, gyroscope x, y, z, and three magnetometer slots that are always zero, since the MPU-6500 has no magnetometer) — a 48-byte datagram, parsed on the host by `glove/protocol/simple.py::SimpleBinaryProtocol.parse` using the `<5H9f` struct format. Frames arrive over UDP (default port 5005) through `glove/transport/udp.py::UDPTransport.recv`, are pushed through `glove/pipeline/main.py::Pipeline.run`, classified, and dispatched to an action binding. On the PC a portable recognition layer classifies the gesture and an OS-specific layer injects it as input (uinput on Linux, SendInput on Windows). The host pipeline is transport- and recogniser-agnostic — `Transport`, `Protocol`, `Recognizer` and `Action` are all interfaces — so the same code runs against the mock transport used in unit tests and the live UDP link.

Deviations from the approved proposal. Two changes were made at the design stage and are stated here so the models below are read against what is built, not what was proposed: (i) the proposed MPU6050 was replaced by the **MPU-6500** (6-axis, I^2C); a 9-axis MPU-9250 was evaluated for its on-board magnetometer, but the fitted part has none, so the three magnetometer slots read zero. (ii) only **two flex sensors** are fitted rather than five — flex sensors are the most expensive part, and two (index + thumb) are sufficient to demonstrate distinct poses for a proof-of-concept. The wire format already carries five flex slots, so the two fitted sensors populate flex[0] and flex[1] and the remaining slots read zero; no host change is needed when the other three are added.

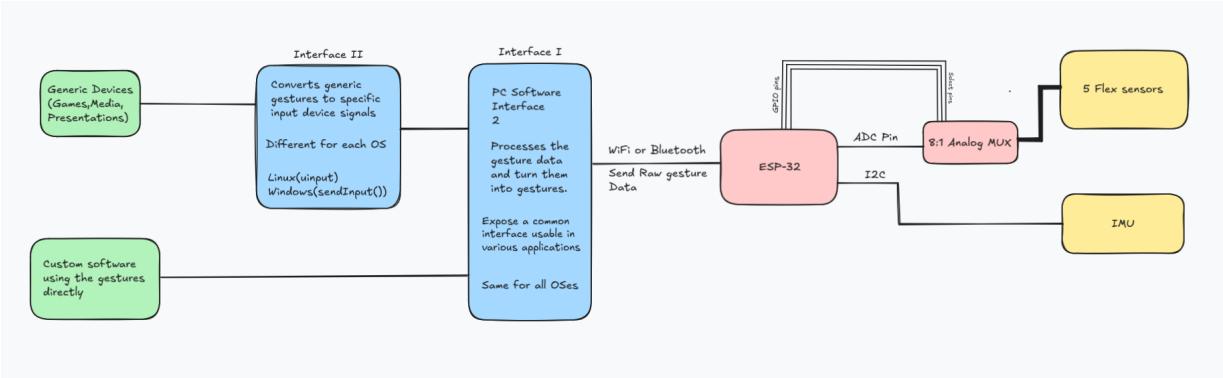


Figure 1: System block diagram: glove sensors → ESP-32 → PC software stack → applications.

1.1 Conditions carried from proposal review

Condition / mandatory addition	Status	Where
Justify the critical function against ≥3 candidates	Done	§2.1
Provide a CDFG with node costs and the critical path	Done	§2.2–2.4
Provide a system FSM with a fault and safe state	Done	§3
State the WCET source and a schedulability proof	Done	§4.1, §4.3

All four conditions from the review sheet are closed; the evidence is the corresponding section.

2 Control Data Flow Graph

2.1 Choosing the critical function

Function / ISR	Trigger	Rate	Est. exec. time	Deadline	Chosen? Why
Window inference (features + model.predict)	prediction timer	4 Hz	∼15 ms	250 ms	**Chosen.** Longest execution, most data dependencies, blocks action dispatch.
Frame poll (wireless RX)	poll	30 Hz	∼2–5 ms	33 ms	No — small, stateless.
Frame parse (SimpleBinaryProtocol)	per datagram	30 Hz	<0.5 ms	33 ms	No — trivial fixed cost.

Justification: window inference in `backend/app/routers/training.py::predict` has the longest execution time, consumes the whole buffered window (most data dependencies), and blocks the action dispatch that follows it.

2.2 The CDFG

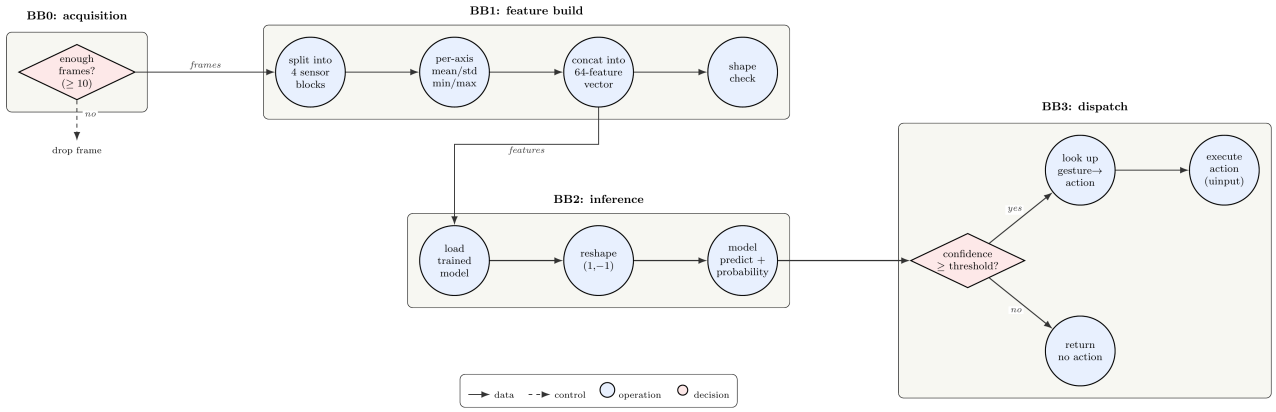


Figure 2: CDFG of `predict()`. Circles = operations, diamonds = decisions, solid = data edges, dashed = control edges, grouped into basic blocks BB0–BB3.

2.3 Node table

Operation	Inputs / output	Est. cost (μs)	Function
Check window ≥10 frames	frame buffer / drop	200	`predict`
Split into 4 sensor blocks	frames / blocks	400	`extract_features`
Per-axis mean/std/min/max	block / 4 stats/axis	2500	`extract_features`
Concat into 64 features	stats / vector	300	`extract_features`
Shape check vs model	features, model / ok,400	100	`predict`
Load pickled model	path / model	1500	`predict`
Reshape (1,−1)	features / 2-D array	50	`predict`
predict + predict_proba	features, model / id, conf	8000	`predict`
Confidence ≥ threshold?	conf, thr / bool	50	`predict_and_act`
Look up action binding	gesture id / record	300	`predict_and_act`
Execute action (uinput)	config / input event	2000	`actions/*`

All costs are estimated (Python-level, dev PC); none measured yet.

2.4 Critical path

Critical path (node sequence)	Total latency	Deadline	Slack
window → split → stats → concat → load → predict → conf → lookup → execute	∼15.3 ms	250 ms	∼234.7 ms

Dependency to break: the model load (1.5 ms) is on the path every call because `predict` re-opens the `.pkl`. *Yes* — cache the model at startup in the Final build. Recomputing full-window statistics (2.5 ms) could be made incremental, but slack is ample, so it is deferred.

3 System-behaviour FSM

3.1 Model type

Mealy. Outputs depend on state *and* input — a recognised gesture emits an action only when confidence ≥ threshold. A Moore machine cannot express "act only if confident".

3.2 State diagram

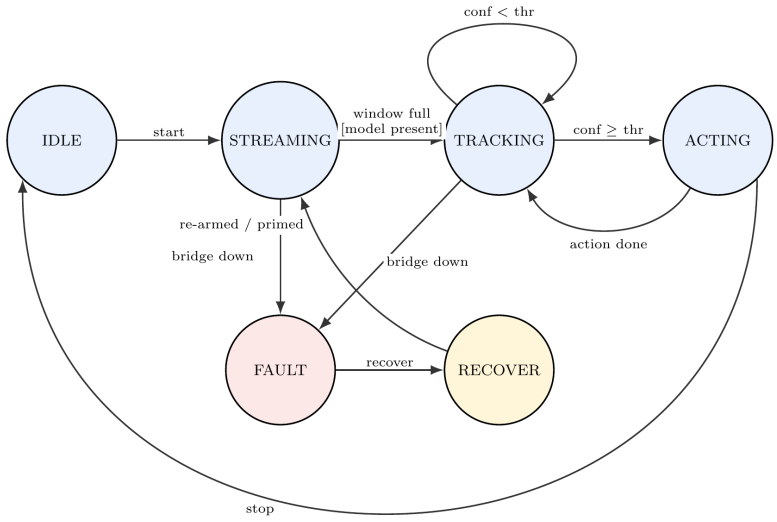


Figure 3: System FSM. Every transition is labelled `event [guard] / action`.

3.3 State transition table

Current state	Event	Guard	Next state	Output / action
IDLE	start	sensors available	STREAMING	open transport, buffer
IDLE	start	sensors unavailable	FAULT	sensor-unavailable error
STREAMING	frame	buffer < window	STREAMING	append frame
STREAMING	window full	bridge up, model present	TRACKING	run inference
STREAMING	link lost	—	FAULT	glove-disconnected error
TRACKING	tick	confidence < threshold	TRACKING	none (display "—")
TRACKING	tick	confidence ≥ threshold	ACTING	dispatch action
ACTING	action done	binding found	TRACKING	inject key/mouse event
ACTING	action done	no binding	TRACKING	none (logged)
FAULT	recover	link/sensor restored	RECOVER	re-arm, clear buffer
FAULT	stop	—	IDLE	close transport
RECOVER	re-armed	buffer primed	STREAMING	resume streaming

3.4 Fault, recovery and safe state

Safe state (FAULT) outputs: the device emits no keyboard or mouse events — no key held, no button pressed, no relative motion; every injected event ends with a release and `syn()`, so a stuck modifier (e.g. held `super`) can never lock the desktop. FAULT is reachable from every other state by the fatal-error / link-loss event. Recovery: `recover → RECOVER`, which clears the frame buffer (stale frames must never be inferred against) and returns to STREAMING only once primed. A watchdog transition (STREAMING/TRACKING → FAULT on link lost) prevents a dropped wireless link leaving the system silently half-running.

3.5 Completeness check

State ↓ / Event →	start	frame	window full	tick / bridge-down
IDLE	STREAMING	—	—	—
STREAMING	—	STREAMING	TRACKING	FAULT
TRACKING	—	—	—	FAULT
ACTING	—	—	—	FAULT
FAULT	—	—	—	RECOVER
RECOVER	—	—	STREAMING	FAULT

4 Task schedule

4.1 Task set

Task	Trigger	Period T	WCET C	Deadl. D	Prio	U	WCET source
IMU sample (`imuTask`)	timer, core 0	10 ms	0.42 ms	10 ms	High	0.042	estimated (I^2C @400 kHz)
Flex sample (`flexTask`)	timer, core 0	10 ms	0.18 ms	10 ms	High	0.018	estimated (ADC 12-bit)
Frame build (`packetTask`)	queue recv	10 ms	0.21 ms	10 ms	Medium	0.021	estimated
Wireless TX (`txTask`)	frame ready	10 ms	0.90 ms	10 ms	Medium	0.090	estimated (UDP sendto)
Host frame poll	timer	33 ms	∼5 ms	33 ms	High	0.152	estimated
Host inference tick	timer	250 ms	15.3 ms	250 ms	Med	0.061	estimated
Watchdog (`wdtTask`)	timer	3000 ms	0.05 ms	3000 ms	Low	0.00002	estimated

Total utilisation U ≈ 0.17 on the ESP-32 (four firmware tasks) **and U ≈ 0.21 on the host** (poll + inference + watchdog). All WCETs are *estimated* from the fixed peripheral costs (I^2C transfer, ADC conversion, UDP `sendto`) until measured with a GPIO toggle and logic analyser.

4.2 Scheduling approach

Task level: Rate-Monotonic (RMS). Operation level: ASAP list scheduling. The task set is small, fixed-rate and preemption-cheap, so RMS static priorities are simple and provable. EDF would allow higher utilisation but needs dynamic sorting and response-time checks; at U ≈ 0.23 it buys nothing. A cyclic executive adds jitter without benefit. ASAP gives the shortest schedule for a mostly-serial CDFG chain.

4.3 Schedulability analysis

RMS utilisation bound $U \le n(2^{1/n}-1)$, $n=3$ (poll, inference, watchdog):

$$\text{bound} = 3(2^{1/3}-1) = 0.7798, \qquad U = \tfrac{5}{33} + \tfrac{15.3}{250} + \tfrac{1}{3000} = 0.2130 \le 0.7798.$$

On the ESP-32 the four firmware tasks run under FreeRTOS with rate-monotonic priorities (periods 10/10/10/3000 ms, $U_{fw} = 0.171$); each completes well inside its 10 ms frame because the samplers are pure blocking I^{2}C/ADC reads with no dynamic allocation. Both task sets are **schedulable** by the utilisation bound. Worst-case response for the host inference task: $R \approx 15.3 + 5$ (one poll interference) $= 20.3$ ms $\ll 250$ ms (slack ≈ 229.7 ms), so no response-time analysis is required. ASAP control-step chain: window → split → stats → concat → load → predict → conf → lookup → execute; all zero mobility except the model load, removable by caching.

4.4 Timeline

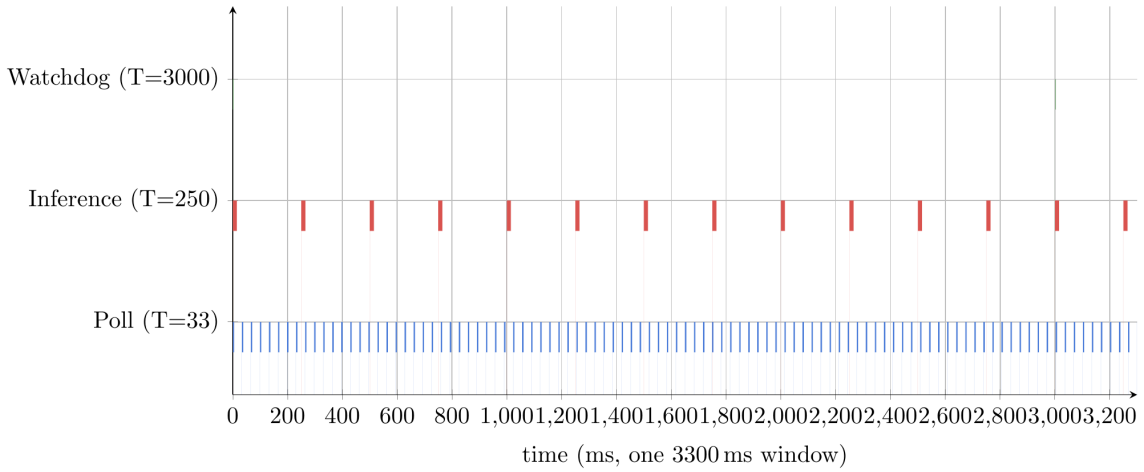


Figure 4: Schedule over one hyperperiod (task level): poll @33 ms, inference @250 ms, watchdog @3000 ms; each inference bar sits well inside its frame.

4.5 Shared resources and interrupts

Shared resource	Used by	Protection	Worst-case blocking
SQLite `glove.db`	record / train / predict	session per request	one transaction; no ISR shares it
Frame buffer	poll (write), inference (read)	single-threaded	none observed
Model `.pkl`	inference, retrain	load-per-request	∼1.5 ms (removed by caching)
uinput device	action dispatch	one UInput/action; `syn()`	serialised by OS input layer
I^{2}C bus	`imuTask` / `flexTask`	one bus mutex	one transaction
Wireless RX ring buffer	firmware `txTask` / host poll	single-producer ring	one buffer copy

5 Planned traceability

Model element	Planned file	Planned function	Owner
Window guard	`PredictUI.js`	runPrediction	DJ
Feature build	`training.py`	extract_features	DJ
Inference	`training.py`	predict	DJ
Dispatch	`training.py`	predict_and_act	DJ
Input injection	`input/keyboard.py`, `mouse.py`	tap / click	DJ
FSM states	`stateMachine.js` (new)	—	SA
Frame poll task	`backend/app/bridge.py`	get_latest_grouped	SA
Inference tick task	`PredictUI.js`	setInterval(...,250)	DJ
Watchdog task	`frontend/src/components/RecordingUI.js`	recordingUI	SA
IMU sampling task	`firmware/src/imu.c`	`imuTask()`	DJ
Flex ADC task	`firmware/src/flex.c`	`flexTask()`	DJ
Frame build task	`firmware/src/packet.c`	`packetTask()`	DJ
Wireless TX task	`firmware/src/net.c`	`txTask()`	SA
Watchdog task	`firmware/src/wdt.c`	`wdtTask()`	SA

6 Plan to Final

Remaining work	Owner	Target date	Risk and fallback
Cache model in memory	DJ	early Oct	Low; fallback = load per request
Extract FSM module	SA	early Oct	Medium; fallback = FSM stays implicit
Integrate ESP-32 + MPU-6500 + flex on the glove	DJ	mid Oct	High (hardware); fallback = IMU-only
FreeRTOS task skeleton (`imuTask`/`flexTask`/`txTask`)	DJ	mid Oct	Medium; fallback = single super-loop
Measure firmware WCETs (GPIO toggle + logic analyser)	DJ	mid Oct	Medium; fallback = keep estimates
Fault-state unit tests	SA	late Oct	Low

7 Individual contribution

Member	Model / section produced	Module for Final	Share %
Dayakrishn Joshi	System recap, CDFG, task set	ESP-32 firmware + ADC acquisition	50
Syed Mohammad Ayaan	System FSM, schedulability, traceability, plan	ESP-32 WiFi/UDP link + PC recognition pipeline	50

References

1. Espressif Systems, *ESP32 Series Datasheet*, 2024.

2. InvenSense, *MPU-6500 Six-Axis (Gyro + Accelerometer) MEMS MotionTracking Device Datasheet*, 2016.
3. InvenSense, *MPU-6500 Register Map and Descriptions*, rev. 1.4, 2016.
4. C. L. Liu and J. W. Layland, "Scheduling Algorithms for Multiprogramming in a Hard-Real-Time Environment," *J. ACM*, 20(1):46–61, 1973.